

# Guía de Versionado y Despliegue en GitHub para este Proyecto

---

## Objetivo

Tener este proyecto:

- versionado con **Git**
- subido a **GitHub**
- desplegado automáticamente
- preparado para seguir evolucionándolo de forma controlada

Esta guía está pensada específicamente para este proyecto en:

**C:\Users\bernat.duran\Documents\Bernat\17.IA\11. Projects IA\Web\_Japó\V0**

---

## Tus datos concretos

Para este caso, tus datos reales son:

- Usuario de GitHub: **BernatDuran**
- URL de perfil: <https://github.com/BernatDuran>
- Email de GitHub para commits: **bernat.duran.mascorda@gmail.com**
- Nombre recomendado del repositorio: **BernatDuran.github.io**
- URL final esperada de publicación: <https://BernatDuran.github.io/>

Por tanto, siempre que en una guía genérica veas algo como:

- **<tu-usuario>**
- **TU-USUARIO**
- **BernatDuran.github.io**

en tu caso real debes leerlo como:

- **BernatDuran**
  - **BernatDuran.github.io**
- 

## 1. Recomendación para este proyecto

La opción más simple y segura para este caso es:

1. Crear un repositorio público llamado: **BernatDuran.github.io**
2. Subir este proyecto ahí.
3. Desplegarlo con **GitHub Pages**.
4. Automatizar el despliegue con **GitHub Actions**.

Por qué esta opción es la mejor

Este proyecto usa rutas absolutas como:

- `/planner.html`
- `/admin.html`
- `/city.html`
- `/icon.svg`

Eso funciona muy bien si la web se publica en la raíz del dominio:

`https://BernatDuran.github.io/`

Si el repositorio tuviera otro nombre, por ejemplo `web-japon`, la web viviría en:

`https://BernatDuran.github.io/web-japon/`

En ese caso habría que adaptar el `base path` de `Vite` y revisar rutas internas.

Para evitar esa complejidad al principio, la mejor práctica aquí es usar un repositorio con nombre:

`BernatDuran.github.io`

---

## 2. Qué significan Git, GitHub, CI y CD

### Git

Es el sistema de control de versiones local. Sirve para:

- guardar el historial del proyecto
- volver atrás si algo se rompe
- entender qué ha cambiado y cuándo

### GitHub

Es la plataforma remota donde subes tu repositorio. Sirve para:

- tener copia en la nube
- colaborar
- gestionar ramas y cambios
- automatizar despliegues

### CI

#### Continuous Integration

Consiste en comprobar automáticamente que el proyecto sigue funcionando técnicamente.

En este caso, lo mínimo sería verificar que:

```
npm run build
```

funciona correctamente.

## CD

### Continuous Deployment

Consiste en desplegar automáticamente la web cuando subes cambios válidos.

---

## 3. Estado actual del proyecto

Este proyecto:

- usa **Vite**
- es multipágina
- genera salida estática en **dist/**
- encaja bien con **GitHub Pages**

Actualmente los scripts principales son:

```
{
  "dev": "vite --configLoader native --host localhost --port 5173 --strictPort",
  "build": "vite build --configLoader native",
  "preview": "vite preview --configLoader native --host localhost --port 4173 --strictPort"
}
```

Y el **.gitignore** ya evita subir:

- **node\_modules**
- **dist**
- logs
- archivos de editor

Eso está bien.

---

## 4. Plan exacto para este proyecto

### Fase 1. Preparar Git localmente

#### Objetivo

Empezar a versionar el proyecto en tu ordenador.

#### Paso 1. Configurar Git en tu PC

Haz esto una sola vez:

```
git config --global user.name "Bernat Duran"  
git config --global user.email "bernat.duran.mascorda@gmail.com"
```

Puedes comprobarlo con:

```
git config --global --list
```

## Paso 2. Inicializar Git en este proyecto

Desde la carpeta del proyecto:

```
git init -b main
```

## Paso 3. Crear el primer commit

```
git add .  
git commit -m "feat: initial project import"
```

## Resultado esperado

- el proyecto queda versionado localmente
- tendrás una rama principal llamada **main**
- ya podrás consultar historial y volver atrás

---

## Fase 2. Crear el repositorio en GitHub

### Objetivo

Crear el contenedor remoto del proyecto.

### Qué debes hacer en GitHub

1. Inicia sesión en [GitHub](#)
2. Pulsa **New repository**
3. Crea un repositorio con este nombre:

**BernatDuran.github.io**

### Ajustes recomendados

- Visibilidad: **Public**
- No marcar:

- Add a README file
- Add .gitignore
- Choose a license

Debe crearse vacío.

Por qué no hay que crear README desde GitHub

Porque ya tienes el proyecto local y es mejor evitar conflictos innecesarios en el primer push.

---

## Fase 3. Conectar el proyecto local con GitHub

### Objetivo

Vincular la carpeta local con el repositorio remoto.

Cuando GitHub te muestre la URL del repositorio, ejecuta:

```
git remote add origin https://github.com/BernatDuran/BernatDuran.github.io.git
git push -u origin main
```

Qué hace cada comando

**git remote add origin ...**

Asocia tu proyecto local con el repositorio remoto de GitHub.

**git push -u origin main**

Sube la rama **main** por primera vez y deja configurado el seguimiento para futuros pushes.

Resultado esperado

- el código ya estará en GitHub
  - la rama **main** existirá en remoto
- 

## Fase 4. Activar despliegue automático con GitHub Pages

### Objetivo

Que GitHub publique automáticamente la web al subir cambios.

#### Paso 1. Activar Pages

En GitHub:

1. Ve al repositorio
2. Entra en **Settings**

3. Ve a **Pages**
4. En **Build and deployment**
5. En **Source**, selecciona:

## GitHub Actions

---

# Fase 5. Crear el workflow de despliegue

## Objetivo

Definir la automatización que construye y publica la web.

Debes crear este archivo:

C:\Users\bernat.duran\Documents\Bernat\17.IA\11. Projects  
IA\Web\_Japó\V0\.github\workflows\deploy.yml

Contenido recomendado:

```
name: Deploy static content to Pages

on:
  push:
    branches: ["main"]
    workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: "pages"
  cancel-in-progress: true

jobs:
  deploy:
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node
        uses: actions/setup-node@v6
        with:
          node-version: lts/*
          cache: npm
```

```
- name: Install dependencies
  run: npm ci

- name: Build
  run: npm run build

- name: Setup Pages
  uses: actions/configure-pages@v6

- name: Upload artifact
  uses: actions/upload-pages-artifact@v5
  with:
    path: ./dist

- name: Deploy to GitHub Pages
  id: deployment
  uses: actions/deploy-pages@v5
```

## Qué hace este workflow

1. Se ejecuta al hacer **push** a **main**
2. Instala dependencias
3. Ejecuta el build
4. Publica la carpeta **dist**

---

## Fase 6. Subir el workflow

### Objetivo

Activar el primer despliegue.

Después de crear **deploy.yml**:

```
git add .github/workflows/deploy.yml
git commit -m "ci: add GitHub Pages deployment workflow"
git push
```

### Resultado esperado

GitHub ejecutará automáticamente el workflow en la pestaña **Actions**.

Cuando termine correctamente, la web estará publicada en:

<https://BernatDuran.github.io/>

Y también podrás abrir:

- <https://BernatDuran.github.io/>

- <https://BernatDuran.github.io/planner.html>
  - <https://BernatDuran.github.io/admin.html>
  - <https://BernatDuran.github.io/city.html?id=...>
- 

## Fase 7. Flujo de trabajo recomendado al principio

### Objetivo

Trabajar de forma controlada, pero sin complicarte demasiado.

Durante la primera etapa, te recomiendo este flujo:

1. Trabajar sobre `main`
2. Hacer cambios pequeños
3. Comprobar build local antes de subir
4. Hacer commits con nombres claros

### Flujo simple recomendado

```
npm run build
git add .
git commit -m "feat: describe el cambio"
git push
```

### Ejemplos de mensajes de commit

- `feat: add planner map view`
  - `fix: restore planner icon encoding`
  - `style: reduce priority icon size`
  - `docs: add GitHub deployment guide`
- 

## Fase 8. Evolución recomendada cuando ya te sientas cómodo

### Objetivo

Profesionalizar el flujo poco a poco.

Cuando ya tengas soltura, el siguiente paso sería trabajar con ramas:

```
git checkout -b feature/planner-toasts
```

Y después:

```
git push -u origin feature/planner-toasts
```



Luego puedes abrir un [Pull Request](#) en GitHub para revisar antes de fusionar a [main](#).

## Ventajas

- cambios más ordenados
  - menor riesgo
  - historial más limpio
- 

## Fase 9. CI mínimo recomendado

### Objetivo

Evitar subir cambios que rompan el proyecto.

El nivel mínimo suficiente para este proyecto es:

1. Workflow de deploy a Pages
2. Validación con `npm run build`

Más adelante puedes añadir otro workflow, por ejemplo:

[.github/workflows/ci.yml](#)

que haga:

```
name: CI

on:
  push:
    branches: ["main"]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with:
          node-version: lts/*
          cache: npm
      - run: npm ci
      - run: npm run build
```

### Qué aporta

- detecta errores antes del despliegue
  - mejora la fiabilidad del proyecto
-

## Fase 10. Protección de rama

### Objetivo

Evitar errores cuando el proyecto ya tenga más valor.

No hace falta activarlo hoy, pero más adelante te interesará:

1. Ir a **Settings** -> **Branches**
2. Proteger la rama **main**
3. Activar:
  - **Require a pull request before merging**
  - **Require status checks to pass before merging**

### Cuándo hacerlo

Cuando ya estés usando ramas y pull requests con normalidad.

---

## 5. Riesgos y detalles específicos de este proyecto

### 1. Rutas absolutas

Como el proyecto usa rutas absolutas, publicar en **BernatDuran.github.io** evita tener que ajustar **base** en **Vite**.

### 2. Proyecto multipágina

Esto no es un problema para GitHub Pages. **Vite** ya construye correctamente:

- **index.html**
- **planner.html**
- **admin.html**
- **city.html**
- **tools.html**

### 3. Texto del manifest PWA

Conviene revisar más adelante el texto del manifest en **vite.config.js**, porque puede haber caracteres raros en:

- **name**
- **short\_name**
- **description**

No bloquea GitHub Pages, pero sí puede afectar a la instalación de la PWA o a textos visibles del manifiesto.

---

## 6. Orden exacto recomendado para empezar

Si quieres hacerlo de la forma más limpia posible, sigue este orden:

1. Crear el repositorio `BernatDuran.github.io`
  2. Configurar Git localmente
  3. Inicializar Git en esta carpeta
  4. Hacer primer commit
  5. Conectar el remoto
  6. Hacer primer push
  7. Crear `deploy.yml`
  8. Activar `GitHub Pages` con `GitHub Actions`
  9. Hacer push del workflow
  10. Ver la web publicada
- 

## 7. Checklist resumido

### Preparación

- ☐ Tener cuenta en GitHub
- ☐ Tener `git` instalado
- ☐ Configurar `user.name`
- ☐ Configurar `user.email`

### Repo

- ☐ Crear repo público `BernatDuran.github.io`
- ☐ No crear README desde GitHub

### Git local

- ☐ `git init -b main`
- ☐ `git add .`
- ☐ `git commit -m "feat: initial project import"`

### Subida

- ☐ `git remote add origin ...`
- ☐ `git push -u origin main`

### Despliegue

- ☐ Activar `GitHub Actions` en Pages
  - ☐ Crear `.github/workflows/deploy.yml`
  - ☐ Commit y push del workflow
  - ☐ Verificar despliegue
- 

## 8. Comandos útiles del día a día

### Ver estado

```
git status
```

Ver historial resumido

```
git log --oneline
```

Añadir todo

```
git add .
```

Crear commit

```
git commit -m "feat: describe el cambio"
```

Subir cambios

```
git push
```

Traer cambios remotos

```
git pull
```

---

## 9. Qué haría yo a continuación

El siguiente paso más práctico para este proyecto sería:

1. Crear el repositorio [BernatDuran.github.io](https://github.com/BernatDuran) en GitHub
2. Inicializar Git en esta carpeta
3. Crear el workflow de deploy
4. Hacer el primer push

---

## 10. Fuentes oficiales

- GitHub Pages: <https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages>

- Crear repositorio: <https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-new-repository>
- Configurar Git: <https://docs.github.com/en/get-started/git-basics/set-up-git>
- GitHub Pages con workflows: <https://docs.github.com/es/pages/getting-started-with-github-pages/using-custom-workflows-with-github-pages>
- Deploy de Vite: <https://vite.dev/guide/static-deploy.html>